04
Oggetti e Classi pt.2
inizializzazione, accessi, distruzione

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2020/2021

Outline

Goal della lezione

- Completare i meccanismi OO di Java a livello di classe
- Porre le basi per poter fare della corretta progettazione
- Discutere aspetti collegati alla vita di un oggetto (costruzione, accesso, distruzione)

Argomenti

- Codice statico
- Costruttori
- Overloading di metodi
- Approfondimento sui package
- Controllo d'accesso
- Finalizzazione e garbage collection

Outline

1. Codice statico
2. Costruttori
3. Controllo d'accesso
4. Distruzione oggetti
5. Una applicazione

Codice statico

Meccanismo

- Alcuni campi e metodi di una classe possono essere dichiarati `static`
- Esempi visti:
  - il `main` dal quale parte un programma Java
  - il metodo `java.util.Arrays.toString()` (ossia, il metodo `toString` nella classe `Arrays` dentro al package `java.util`)
  - il campo `java.lang.System.out`
  - il metodo `java.lang.System.getProperty()`
- Tali campi e metodi sono considerate proprietà della classe, o comunque dell'insieme di tutti gli oggetti, non di un singolo oggetto
- Sintassi: `classe.metodo(argomenti)`, e `classe.campo`
  - omettendo la classe si assume quella da cui parte la chiamata

Sulla notazione

- Iniziano con minuscolo: nomi di package, metodi e campi
- Iniziano con maiuscolo: nomi di classe

⇒ Questo consente di capire facilmente il significato delle istruzioni

Uso di codice statico o non statico in una classe C?

Codice non statico (detto anche codice istanza)

- È codice puro object-oriented
- Definisce le operazioni e lo stato di ogni oggetto creato da C

Codice statico

- Non è codice puro object-oriented, ma codice in stile "imperativo/strutturato"
- Definisce funzioni e variabili del componente software definito da C
  - possono essere visti come metodi e campi dell'unico oggetto "classe C"

Usarli assieme?

- Le helper class abbiano solo metodi/campi statici
- Le classi OO non abbiano metodi/campi statici, a meno di qualche funzionalità "generale" della classe (si veda nel prosieguo)

Esempio Point3D

```
class Point3D { // dichiarazione classe
  double x; // 3 campi
  double y;
  double z;

  void build(double a, double b, double c) {
    this.x = a;
    this.y = b;
    this.z = c;
  }

  double getModuloQuadro() {
    return this.x * this.x + this.y * this.y + this.z * this.z;
  }

  static Point3D zero = new Point3D(); // 0,0,0

  static Point3D max(Point3D[] ps) { // metodo statico
    Point3D max = zero; // ricerca max
    for (Point3D elem : ps) {
      if (elem.getModuloQuadro() > max.getModuloQuadro()) {
        max = elem;
      }
    }
    return max;
  }
}
```

Uso Point3D

```
class UsePoint3D {
  public static void main(String[] s) {
    // creo e inizializzo vari punti
    Point3D p1 = new Point3D();
    p1.build(10, 20, 30);
    Point3D p2 = new Point3D();
    p2.build(5, 6, 7);
    Point3D p3 = new Point3D();
    p3.build(100, 100, 100);
    Point3D p4 = Point3D.zero; // questo è lo zero

    // costruisco l'array
    Point3D[] array = new Point3D[] { p1, p2, p3, p4 };

    // calcolo il max
    Point3D max = Point3D.max(array);

    // stampo
    System.out.println("Max: " + max.x + "," + max.y + "," + max.z);
  }
}
```

Point3D: commenti

Si notino le due diverse chiamate

- `build` è sempre chiamata su un oggetto, ossia su: `new Point3D()`
- `zero` e `max` sono chiamate sulla classe `Point3D`

Razionale

- `max` è una funzionalità sui `Point3D`, quindi è naturale includerla nella classe `Point3D`, ma non è inquadrabile come funzionalità di un singolo oggetto (ognuno darebbe la stessa risposta)
- `zero` è una sorta di "costante", quindi non è proprietà di un oggetto

Un errore comune

- se si omette l'indicazione del receiver, si rischia di chiamare una proprietà non statica da un metodo statico, e questo comporta un errore segnalato dal compilatore

$\Rightarrow$ a questo si ovvia inserendo sempre l'indicazione del receiver

Variante Point3DBis.. inizializzazione con ritorno

```
class Point3DBis { // dichiarazione classe
  double x; // 3 campi
  double y;
  double z;

  Point3DBis build(double a, double b, double c) { // build con ritorno
    this.x = a;
    this.y = b;
    this.z = c;
    return this;
  }

  double getModuloQuadro() {
    return this.x * this.x + this.y * this.y + this.z * this.z;
  }

  static Point3DBis zero = new Point3DBis().build(0, 0, 0);

  static Point3DBis max(Point3DBis[] ps) { // metodo statico
    Point3DBis max = zero; // ricerca max
    for (Point3DBis elem : ps) {
      if (elem.getModuloQuadro() > max.getModuloQuadro()) {
        max = elem;
      }
    }
    return max;
  }
}
```

Uso Point3DBis

```
class UsePoint3DBis {
  public static void main(String[] s) {
    // creo vari punti
    Point3DBis p1 = new Point3DBis().build(10, 20, 30);
    Point3DBis p2 = new Point3DBis().build(5, 6, 7);
    Point3DBis p3 = new Point3DBis().build(100, 100, 100);
    Point3DBis p4 = Point3DBis.zero; // questo è lo zero

    // costruisco l'array
    Point3DBis[] array = new Point3DBis[] { p1, p2, p3, p4 };

    // calcolo il max
    Point3DBis max = Point3DBis.max(array);

    // stampo
    System.out.println("Max: " + max.x + "," + max.y + "," + max.z);
  }
}
```

Point3DBis: commenti

Si noti la struttura del metodo `build`

- inizializza il valore dei campi
- restituisce `this`

Razionale

- è possibile concatenare creazione di oggetto e sua inizializzazione
- schema chiamato "fluente"

Sull'uso delle proprietà `static`

Consigli generali

- nella programmazione OO pura, andrebbero interamente evitati
- frequente invece l'uso di costanti
- a volte sono comodi, per ora cercare di tenere separate nelle classi le parti `static` dalle altre (poi vedremo "pattern" più sofisticati)
- le librerie di Java ne fanno largo uso

Prassi frequente in Java

- se `XYZ` è una classe usata per generare oggetti, la classe `XYZs` conterrà solo proprietà statiche relative
- es: `Object/Objects`, `Array/Arrays`, `Collection/Collections`

In altri linguaggi.. come Scala

- parte statica e non-statica vanno necessariamente posizionate in porzioni diverse del codice

Ristrutturazione Point3D

```
class Point3D {
  double x;
  double y;
  double z;

  Point3D build(double a, double b, double c) {
    this.x = a;
    this.y = b;
    this.z = c;
    return this;
  }

  double getModuloQuadro() {
    return this.x * this.x + this.y * this.y + this.z * this.z;
  }

}
```

Modulo Points

```
class Points { // Modulo con funzionalità per punti

  static Point3D zero = new Point3D().build(0, 0, 0);

  static Point3D max(Point3D[] ps) { // metodo statico
    Point3D max = zero; // ricerca max
    for (Point3D elem : ps) {
      if (elem.getModuloQuadro() > max.getModuloQuadro()) {
        max = elem;
      }
    }
    return max;
  }
}
```

Uso Point3D e Points

```
class UsePoint3D {
  public static void main(String[] s) {
    // creo vari punti
    Point3D p1 = new Point3D().build(10, 20, 30);
    Point3D p2 = new Point3D().build(5, 6, 7);
    Point3D p3 = new Point3D().build(100, 100, 100);
    Point3D p4 = Points.zero; // questo è lo zero

    // costruisco l'array
    Point3D[] array = new Point3D[] { p1, p2, p3, p4 };

    // calcolo il max
    Point3D max = Points.max(array);

    // stampo
    System.out.println("Max: " + max.x + "," + max.y + "," + max.z)
    ;
  }
}
```

Outline

1. Codice statico
2. Costruttori
3. Controllo d'accesso
4. Distruzione oggetti
5. Una applicazione

La costruzione degli oggetti

L'operatore `new`

- Allo stato attuale delle nostre conoscenze, crea un oggetto inizializzando tutti i suoi campi al loro valore di default (p.e. 0 per i numeri), e ne restituisce il riferimento
- Altri metodi devono gestire la inizializzazione vera e propria
- Problema: garantire una corretta inizializzazione

Costruttori di una classe

- Assomigliano per struttura ai metodi
- Hanno lo stesso nome della classe in cui si trovano
- Nessun tipo di ritorno, possono avere dei parametri formali
  - ⇒ alla `new` si possono quindi passare dei valori
- Il costruttore di default (a zero argomenti) è implicitamente definito solo se non se ne aggiungono altri – ecco perché era consentito scrivere: `new Point3D()`

Esempio: Point3D

```
class Point3D {   // dichiarazione classe
    double x;
    double y;
    double z;
    Point3D(double inx,double iny,double inz){ //costruttore
        this.x = inx; // metto l'argomento inx in this.x
        this.y = iny; // ..
        this.z = inz; // ..
    }
    ...
}

//creo l'oggetto usando il costruttore a tre argomenti
Point3D p = new Point3D(10.0,20.0,30.0);
// stampo
System.out.println("p: " + p.x + "," + p.y + "," + p.z);
// costruttore di "default" in questo caso non funziona!
//Point3D p2=new Point3D(); NO!!
```

Gli argomenti del costr. spesso corrispondono ai campi

```
class Point3D {   // dichiarazione classe
    double x;
    double y;
    double z;
    Point3D(double x,double y,double z){
        this.x = x; // metto l'argomento x in this.x
        this.y = y; // ..
        this.z = z; // ..
    }
    ...
}

//creo l'oggetto usando il costruttore a tre argomenti
Point3D p = new Point3D(10.0,20.0,30.0);
// stampo
System.out.println("p: " + p.x + "," + p.y + "," + p.z);
```

Esempio: Persona (3 costruttori)

```
class Persona { // dichiarazione classe
  static int currentYear = new java.util.Date().getYear();
  String nome;
  int annoNascita;
  boolean sposato;

  Persona(String nome) {
    this.nome = nome;
    this.annoNascita = Persona.currentYear;
    this.sposato = false;
  }

  Persona(String nome, int annoNascita) {
    this.nome = nome;
    this.annoNascita = annoNascita;
    this.sposato = false;
  }

  Persona(String nome, int annoNascita, boolean sposato) {
    this.nome = nome;
    this.annoNascita = annoNascita;
    this.sposato = sposato;
  }
}
```

Esempio: Uso di Persona (3 costruttori)

```
class UsePersona {
  public static void main(String[] s) {
    // Persona p1=new Persona(); NO!!
    Persona p2 = new Persona("Mario Rossi");
    Persona p3 = new Persona("Gino Bianchi", 1979);
    Persona p4 = new Persona("Carlo Verdi", 1971, true);
  }
}
```

Sequenza d'azioni effettuate con una `new`

- si crea l'oggetto con tutti i campi inizializzati al default
- si esegue il codice del costruttore (`this` punta all'oggetto creato)
- la `new` restituisce il riferimento `this`

Istruzione this(..)

Usabile per chiamare un altro costruttore

- tale istruzione può solo essere la prima di un costruttore
- questo meccanismo consente di riusare il codice di altri costruttori

```
class Persona2 { // dichiarazione classe
  static int currentYear = new java.util.Date().getYear();
  String nome;
  int annoNascita;
  boolean sposato;

  Persona2(String nome, int annoNascita, boolean sposato) {
    this.nome = nome;
    this.annoNascita = annoNascita;
    this.sposato = sposato;
  }

  Persona2(String nome, int annoNascita) { // richiama costruttore a 3 arg..
    this(nome,annoNascita,false);
  }

  Persona2(String nome) {
    this(nome,Persona2.currentYear); // richiama costruttore a 2 arg..
  }

}
```

Overloading dei costruttori

Overloading

- È la tecnica che consente di dare più significati ad uno stesso "nome"
- L'overloading è un meccanismo importante per il programmatore
- La difficoltà che comporta è dovuta alla tecnica di disambiguazione
- Due esempi visti finora: overloading operatori matematici, overloading costruttori

Overloading dei costruttori

Data una `new`, quale costruttore richiamerà? (JLS 15.12)

- si scartano i costruttori con numero di argomenti che non corrisponde
- si scartano i costruttori il cui tipo d'argomenti non è compatibile
- se ve ne è uno allora è quello che verrà chiamato..
- altrimenti il compilatore segnala un errore

Overloading dei metodi

Overloading dei metodi

- Si può fare overloading dei metodi con stessa tecnica di risoluzione
- Sia su metodi statici, che su metodi standard (metodi istanza)

```
class ExampleOverloading {
  static int m(double a, int b) {
    return 1;
  }

  static int m(int a, double b) {
    return 2;
  }

  static int m2(double a, double b) {
    return 1;
  }

  static int m2(int a, int b) {
    return 2;
  }

  public static void main(String[] s) {
    // System.out.println(""+m(1,1)); AMBIGUOUS!
    // System.out.println(""+m(1.5,1.5)); NO COMPATIBLE!

    System.out.println("" + m2(1.5, 1.5)); // 1
    System.out.println("" + m2(1, 1)); // 2
  }
```

Outline

1. Codice statico
2. Costruttori
3. Controllo d'accesso
4. Distruzione oggetti
5. Una applicazione

I package

Problema

- Un framework mainstream come quello di Java può disporre di decine di migliaia di classi di libreria e di applicazione
- È necessario un meccanismo per consentire di strutturarle in gruppi (a più livelli gerarchici)
- Per poter meglio gestirli in memoria secondaria, e per meglio accedervi

Package in Java

- Ogni classe appartiene ad un package
- Un package ha nome gerarchico `n1.n2....nj` (p.e. `java.lang`)
- Le classi di un package devono trovarsi in una medesima directory
- Questa è la subdirectory `n1/n2/../nj` a partire da una delle directory presenti nella variabile d'ambiente `CLASSPATH`

I package pt. 2

Dichiarazione del package

- Ogni unità di compilazione (file `.java`) deve specificare il suo package
- Lo fa all'inizio del sorgente con la direttiva `package pname;`
- Se non lo fa, allora trattasi del package di "default"
- Se lo fa, tale file dovrà stare nella opportuna directory

Importazione classi da altri package

- Una classe va usata (p.e. nella `new` o come tipo) specificando anche il package (p.e. `new java.util.Date()`)
- Per evitare tale specifica, si inserisce una direttiva di importazione
  - `import java.util.*;` importa tutte le classi di `java.util`
  - `import java.util.Date;` importa la sola `java.util.Date`

Unità di compilazione e livello d'accesso "package"

Unità di compilazione

- È un file compilabile in modo atomico da `javac`
- Si deve chiamare con estensione `.java`
- Può contenere varie classi indicate una dopo l'altra

Livello d'accesso package

- Le classi in una unità di compilazione, i loro metodi, campi, e costruttori hanno di default il livello d'accesso package

⇒ sono visibili e richiamabili solo dentro al package stesso

⇒ sono invisibili da fuori

Livelli d'accesso public e private

Livello d'accesso public – visibile da tutte le classi

- Lo si indica anteponendo alla classe/metodo/campo/costruttore la keyword public
- La corrispondente classe/metodo/campo/costruttore sarà visibile ed utilizzabile da qualunque classe, senza limitazioni

Livello d'accesso private – visibile solo nella classe corrente

- Lo si indica anteponendo al metodo/campo/costruttore la keyword private
- Il corrispondente metodo/campo/costruttore sarà visibile ed utilizzabili solo dentro alla classe in cui è definito

Qualche conseguenza

A livello di classe

- In una unità di compilazione solo una classe può essere `public`, e questa deve avere lo stesso nome del file `.java`

A livello di metodi/campi/costruttori

- la scelta `public`/`private` può consentire di gestire a piacimento il concetto di information hiding, come approfondiremo la prossima settimana

La keyword `final`

`final` = non modificabile

- Ha vari utilizzi: in metodi, campi, argomenti di funzione e variabili
- Tralasciamo per ora il caso dei metodi
- Negli altri casi denota variabili/campi assegnati e non più modificabili

Il caso più usato: costanti di una classe

- Es.: `public static final int CONST=10;`
- Perché usarle?
  - Anche se `public`, si ha la garanzia che nessuno le modifichi
  - Sono gestibili in modo più performante

Il caso dei "magic number"

- Ogni numero usato in una classe per qualche motivo (3,21,..)..
- ..sarebbe opportuno venisse mascherato da una costante, per motivi di leggibilità e di più semplice modificabilità

Esempio Magic Numbers

```
public class MagicExample {
  // Put 100 into a constant and give it a name!!
  private static final int SIZE = 100;

  public static void main(String[] s) {
    double[] array = new double[SIZE];
    double sum = 0;
    for (int i = 0; i < SIZE; i++) {
      // Assegno un numero random
      array[i] = Math.random();
      sum = sum + array[i];
    }
    System.out.println("Somma " + sum);
  }
}
```

GuessMyNumberApp revisited

```java
import java.io.Console; // Classe non funzionante dall'IDE Eclipse
import java.util.Random;

public class GuessMyNumberApp {

  public static final int ATTEMPTS = 10;
  public static final int MAX_GUESS = 100;
  public static final int MIN_GUESS = 1;

  public static void main(String[] args) {
    int number = new Random().nextInt(MAX_GUESS - MIN_GUESS) + MIN_GUESS;
    for (int i = 1; i <= ATTEMPTS; i++){
      System.out.println("Attempt no. "+i);
      System.out.println("Insert your guess.. ");
      int guess = Integer.parseInt(System.console().readLine());
      if (guess == number){
        System.out.println("You won!!");
        return;
      } else if (guess > number){
        System.out.println("Your guess is greater..");
      } else {
        System.out.println("Your guess is lower..");
      }
    }
    System.out.println("Sorry, you lost!");
  }
}
```

Outline

1. Codice statico
2. Costruttori
3. Controllo d'accesso
4. Distruzione oggetti
5. Una applicazione

Allocazione degli oggetti in memoria

Operatore `new`

- Incorpora l'equivalente della funzione `malloc` del C
- Il compilatore calcola la dimensione necessaria da allocare
- La `new` chiama il gestore della memoria, che alloca lo spazio necessario
- Si inizializza l'area e si restituisce il suo riferimento

Cosa c'è in un oggetto (e, quanto è grande?)

- Non tutti i dettagli sono noti, dipende dalla JVM
- Di sicuro contiene i seguenti elementi:
  - Spazio per ogni campo (non statico) della classe
  - Un riferimento ad una struttura dati relativa alla classe dell'oggetto
  - Un riferimento alla tabella dei metodi virtuali (una struttura dati necessaria a trovare dinamicamente i metodi da richiamare)

Distruzione degli oggetti

Il tempo di vita degli oggetti

- Durante l'esecuzione di un programma, è verosimile che molto oggetti vengano creati
- Ogni creazione comporta l'uso di una parte di memoria centrale
- Non è noto quanto durerà l'esecuzione del programma

$\Rightarrow$ Qualcuno dovrà preoccuparsi di deallocare la memoria

Il garbage collector (GC)

- E' un componente della JVM richiamato dalla JVM con una frequenza che dipende dallo stato della memoria
- Ogni volta, cerca oggetti in memoria heap che nessuna parte attiva del programma (thread) sta più usando (neanche indirettamente)
- Trovatili, li dealloca (come la `free` del C) senza che il programmatore debba occuparsene

Esempio funzionamento GC

```
class GC {
  private static long size = 1000;

  public static void main(String[] s) {
    // Runtime dà info sull'esecuzione
    Runtime r = Runtime.getRuntime();

    // Creo oggetti all'infinito
    for (long l = 0; true; l++) {
      new Object();
      // Stampo solo ogni tanto
      if (l % size == 0) {
        System.out.print("Objs (*10^6): " + l / 1000000);
        System.out.println(" Freemem (MB):" + (r.freeMemory() >> 20));
      }
      // La memoria libera si vedrà calare lentamente
      // e poi riprendersi di colpo, ciclicamente
    }
  }
}
```

Outline

1. Codice statico
2. Costruttori
3. Controllo d'accesso
4. Distruzione oggetti
5. Una applicazione

Applicazione: Mandelbrot

Problema

Data una semplice classe `Picture` che gestisce gli aspetti grafici, realizzare una applicazione che disegna il frattale Mandelbrot.

Elementi progettuali

- Classe fornita `Picture` – codice non comprensibile ora
  - ha un costruttore che accetta larghezza e altezza in pixel della finestra
  - metodo `void drawPixel(int x,int y,int color)`
- Classe `Complex` modella numeri complessi e operazioni base
- Classe `Mandelbrot` si occupa di calcolare il valore di ogni punto del rettangolo
  - metodo `void advancePosition()` passa al prossimo punto
  - metodo `boolean isCompleted()` dice se ci sono altri punti da calcolare
  - metodo `int computeIteratrions()` dice quante iterazioni vengono calcolate per il punto corrente
- Classe `MandelbrotApp` ha il solo `main`

Elementi implementativi

- Implementazione ancora preliminare e da migliorare

Classe Complex

```
1  public class Complex {
2
3    double re;
4    double im;
5
6    Complex(double re, double im) {
7      this.re = re;
8      this.im = im;
9    }
10
11   double getScalarProduct(){
12     return this.re * this.re + this.im * this.im;
13   }
14
15   // Crea un nuovo complesso, sommando this a c
16   Complex sum(Complex c){
17     return new Complex(this.re + c.re, this.im + c.im);
18   }
19
20   // Crea un nuovo complesso, moltiplicando this a c
21   Complex times(Complex c){
22     return new Complex(this.re * c.re - this.im * c.im,
23                        c.re * this.im + c.im * this.re);
24   }
25
26 }
```

Classe Mandelbrot

```java
public class Mandelbrot {

  static final double MAX_PRODUCT = 4.0;
  int width, height, x, y; // uno per linea di norma
  double minx, maxx, miny, maxy, maxIter; // uno per linea di norma

  Mandelbrot(int width, int height, double minx, double maxx,
         double miny, double maxy, int maxIter) {
    this.width = width; this.height = height;
    this.minx = minx; this.maxx = maxx;
    this.miny = miny; this.maxy = maxy;
    this.maxIter = maxIter;
  }
  void advancePosition(){
    x = (x + 1) % width;
    y = y + (x == 0 ? 1 : 0);
  }
  boolean isCompleted(){
    return y == height;
  }
  int computeIterations(){
    Complex c0 = new Complex(this.minx + (this.maxx - this.minx) * x / width,
                             this.miny + (this.maxy - this.miny) * y / height);
    Complex c = c0;
    int iter;
    for (iter = 0; c.getScalarProduct() < MAX_PRODUCT && iter < this.maxIter; iter++) {
      c = c.times(c).sum(c0); // c = c*c + c0
    }
    return iter;
  }
}
```

Classe MandelbrotApp

```java
public class MandelbrotApp {

  public static final int WIDTH = 800;
  public static final int HEIGHT = 800;
  public static final double MINX = -1.5;
  public static final double MAXX = 0.5;
  public static final double MINY = -1.0;
  public static final double MAXY = 1.0;
  public static final int MAX_ITER = 32;

  public static int greyColorFromIterations(int iter, int maxIter){
    if (iter == MAX_ITER) { // Out of Mandelbrot set
      return 0;
    }
    iter = 255-iter*(256/MAX_ITER); // 255,254,..,0 colors
    return iter | iter << 8 | iter << 16; // as grey

  }

  public static void main(String[] s){

    Mandelbrot mb = new Mandelbrot(WIDTH,HEIGHT,MINX,MAXX,MINY,MAXY,MAX_ITER);
    Picture p = new Picture(WIDTH,HEIGHT);
    while (!mb.isCompleted()){
      int iter = mb.computeIterations();
      int color = greyColorFromIterations(iter,MAX_ITER);
      p.drawPixel(mb.x, mb.y, color);
      mb.advancePosition();
    }
  }

}
```

Preview del prossimo laboratorio

Obbiettivi

- Esercizi su piccoli algoritmi su array e tipi primitivi
- Uso costruttori
- Aspetti avanzati della compilazione in Java